

Introducere în programare - curs și laborator - suport electronic

(C) 2023-2024 Bogdan Pătruț

Meniu

[Lectia 1](#) | [Lectia 2](#) | [Lectia 3](#) | [Lectia 4](#) | [Lectia 5](#) | [Lectia 6](#) | [Lectia 7](#)
[Material pentru laboratoare](#)

Rezumat video



Lectia 1 - Elemente introductive. Expresii, operatori, tipuri de date

Scopul acestui curs este prezentarea unor noțiuni introductive despre compilare/interpretare și recapitularea unor cunoștințe despre elementele de bază din limbajul C/C++ (despre variabile, expresii, tipuri de date, operatori, instrucțiunea de atribuire). Accentul se va pune pe elementele de finețe, pe operatorii mai puțin cunoscuți (inclusiv operatorii pe biți), pe prioritatea și asociativitatea operatorilor, pe tipurile de date și reprezentările lor, pe conversia între tipuri de date și altele.

Video pentru cursul 1:

- Exemplu de programare logică: [click aici](#)
- Compilare, interpretare: [click aici](#)
- Despre alfabetul limbajului: [click aici](#)
- Sintaxa BNF: [click aici](#)
- Alte elemente din cursul 1: [click aici](#)
- Exemplu cu operatori simpli: [click aici](#)

1.1. Programarea calculatoarelor, limbaje de programare, istoric

Programarea calculatoarelor

Programarea este o activitate informatică de elaborare a produselor-program, a programelor (software) necesare.

▼ Definiții

- Programarea conține următoarele subactivități: specificarea, proiectarea, implementarea, documentarea și întreținerea produsului program.
- Este un proces ce conduce de la formularea unei probleme (de calcul) la un program executabil pe un calculator
- Sarcini înrudite sunt testarea, depanarea, întreținerea codului sursă, implementarea sistemului construit, managementul acestuia etc; acestea pot fi considerate ca părți ale procesului de programare, dar cel mai adesea sunt asociate termenului de dezvoltare software
- Ingineria software (ingineria programării) combină tehnici ingineresti cu practici de dezvoltare software
- În ingineria software, programarea (implementarea) este privită ca una din fazele procesului de dezvoltare software
- Există o permanentă dezbateră dacă scrierea de programe este o formă de artă, un meșteșug sau o disciplină inginerască

Există mai multe *paradigme de programare*. O **paradigmă de programare** reprezintă o modalitate de a clasifica limbajele de programare în funcție de caracteristicile lor.

▼ Paradigme de programare

- **Programarea structurată** este o paradigmă a programării apărută după anul 1970 datorită complicării crescânde a programelor de calculatoare. A apărut ca un model nou de programare, în scopul de a crea noi tehnici de programare apte de a produce programe care să fie sigure în funcționare, pe o durată mai lungă. În primii ani ai informaticii, limbajul de programare BASIC nu conținea toate elementele programării structurate, așa încât unele cicluri trebuiau realizate folosind instrucțiunea de salt necondiționat GOTO. Limbajul de programare Pascal este un exemplu de limbaj de programare structurat, conceput special pentru învățarea programării. Astăzi, toate limbajele de programare imperative sunt structurate. Acestea folosesc **structuri de control de bază** (secvența sau structura liniară, decizia cu ambele ramuri (selecția), repetiția condiționată anterior) și **structuri derivate** (decizia cu ramura „altfel” vidă, decizia multiplă, repetiția cu test final sau cu număr cunoscut de pași)

▼ Teorema Böhm și Jacopini

Există o serie de teoreme care se referă la programarea structurată, dar pentru noi are importanță Teorema de structură a lui Böhm și Jacopini, care se poate enunța astfel: orice schemă logică nestructurată poate fi înlocuită cu una echivalentă, dar structurată, prin adăugarea de noi acțiuni și condiții. Teorema se referă la schemele logice, dar se poate aplica și algoritmilor descriși în pseudocod sau chiar scriși sub formă de program, într-un limbaj de programare. Așadar, orice program poate fi pus sub o formă structurată, adică să conțină doar structurile de bază și/sau structurile auxiliare, prin utilizarea unor variabile boolene asociate unor funcții suplimentare.

▼ Exemplificare

Să exemplificăm pe cazul unui algoritm scris în limbaj pseudocod. Algoritmul următor determină cel mai mic element (notat `min`) din șirul `X` de `n` elemente:

```
Citeste(n, X);
i = 2; min = X(1);
A:
dacă i > n atunci
    treci la pasul B
altfel
    dacă X(i) < min atunci
        min = X(i);
    i = i+1;
    treci la pasul A
B:
Scrie(min)
```

Observați că algoritmul nu este scris sub o formă structurată (apare instrucțiunea de salt necondiționat "treci la pasul"). În schimb, următoarea formă reprezintă același algoritm și este structurată:

```
Citeste(n, X);
min = X(1);
pentru i de la 2 la n execută
    dacă X(i) < min atunci
        min = X(i);
Scrie(min)
```

Aceasta este cea mai elegantă formă de scriere a algoritmului de determinare a minimumului. O altă formă, care folosește doar structurile de bază ar fi:

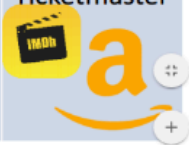
```
Citeste(n, X);
min = X(1);
i = 2;
cât timp i ≤ n execută
    dacă X(i) < min atunci
        min = X(i);
    i = i+1
Scrie(min)
```

- **Programarea imperativă/procedurală**, în contrast cu programarea declarativă, este o paradigmă de programare care descrie calculul ca instrucțiuni ce modifică starea unui program. În aproape același fel în care modul imperativ din limbajele naturale exprimă comenzi pentru acțiuni, programele imperative sunt o secvență de comenzi pentru acționarea calculatorului. Majoritatea limbajelor de programare de care ați auzit sunt limbaje de programare imperative (C, C++, Pascal, JavaScript, Java, Python, Basic, PHP etc.)
- **Programarea declarativă** este un stil de programare în care se descrie logica unui calcul, fără să se prezinte modul de execuție; programarea declarativă răspunde la întrebarea *ce* trebuie calculat și nu la întrebarea *cum* se face acest lucru.
 - **Programarea funcțională** este o paradigmă de programare care tratează calculul ca evaluare de funcții matematice și evită starea și datele muabile. Se pune accent pe aplicarea de funcții, spre deosebire de programarea imperativă, care folosește în principal schimbările de stare. Exemple de limbaje de programare pentru această paradigmă sunt Haskell și CaML.
 - **Programarea logică** este o paradigmă de programare bazată pe logica predicatelor de ordinul I. Folosește conceptele de predicate și de clauză. Reprezentantul acestei paradigme este limbajul de programare PROLOG, care a fost dezvoltat pentru prelucrarea limbajului natural, în inteligența artificială.
 - **Programarea bazată pe reguli** se înscrie în paradigma programării declarative, programul constând din aplicarea unor reguli asupra unor fapte, pentru a genera noi fapte. Ordinea de aplicare a regulilor este necunoscută de programator, iar regulile se aplică până când nu se mai poate aplica niciuna, care să genereze noi fapte. Exemple de limbaje de programare bazate pe reguli sunt CLIPS, EXSYS și se folosesc pentru generarea de sisteme expert, în inteligența artificială.
- **Programarea orientată spre obiecte** (sau programarea orientată-obiect) este o paradigmă de programare, axată pe ideea încapsulării, adică grupării datelor și codului care operează asupra lor, într-o singură structură. Un alt concept important asociat programării orientate obiect este polimorfismul. Astăzi majoritatea limbajelor de programare acceptă această paradigmă, dar la început limbajul de programare C++ a fost un bun exemplu. Plecând de la C++ s-a dezvoltat limbajul de programare Java, în care totul este obiectual.

Puteți viziona aici un videoclip de prezentare a paradigmelor de programare <https://www.youtube.com/watch?v=AmS2-9KEeS0>

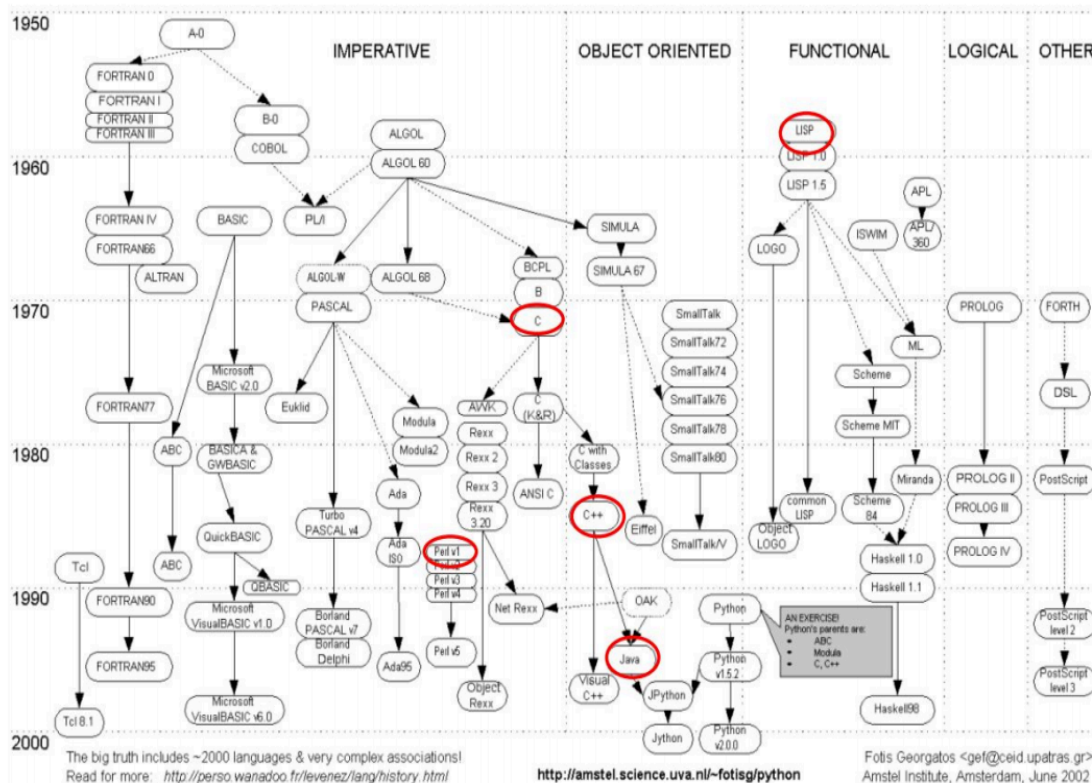
Limbaje de programare

▼ Evoluția limbajelor de programare

1957-1959	FORTRAN (FORmulaTRANslation) LISP (List Processor) COBOL (Common Business-oriented language) Oldest languages still in use High-level	Terminator's vision has samples of COBOL source code	Supercomputing appl., AI devel., business software	NASA ATMs, Credit cards
1970	PASCAL (<- Blaise Pascal) High-level, for teaching data structuring	Niklaus Wirth (Turing '84) 	Teaching programming	
1972	C General-purpose, with many derivatives (C#, Java, Perl, PHP, Python)	Dennis Ritchie (Turing '83, K.Thomson) 	Cross-platform programming, System prog., Unix prog, computer game devel.	UNIX [®] early www servers & clients
1979-1983-...	C++ High-level, OO, expands C	Bjarne Stroustrup 	Commercial appl. devel., embedded software, server/client applic., video games	
1983	Objective C (OO extension of C) High-level, general purpose, OO, expands C with messages based on Smalltalk	Brad Cox & Tom Love 	Apple programming (OS X and iOS) WWDC '14: Swift = Objective C without C WWDC '15: Swift 2.0	
1987	PERL (Practical Extraction and Reporting Language) High-level, general-purpose, interpreted, multi-paradigm language	Larry Wall 	Text processing, graphics programming, system administration, network programming, finance, bioinformatics	Priceline, Ticketmaster 

1989-1991	PYTHON (for Monty Python's Flying Circus) High-level, general purpose, multi-paradigm language	Guido van Rossum 	WAD, software devel., information security, biologists, bioinformatics	Google YAHOO! Spotify
1993	RUBY (a collab's birthstone) High-level, general purpose, OO. Designed (Ada, C++, Perl, Lisp Python) for productive & enjoyable programming	Yukihiro Matsumoto 	Web appl. devel.,	Twitter hulu GROUPON
1995	JAVA (for the coffee consumed) High-level, general-purpose, cross-platform, multi-paradigm language	James Gosling (Sun Microsystems) 	network programming, WAD, GUI devel., software devel.	SAP a ebay
1995	PHP ("Personal Home Page" -> Hypertext Pre-processor) General purpose, open source for building dynamic web pages; influenced by C/C++, Perl, Java	Rasmus Lerdorf 	Building / maintaining dynamic web pages, server side devel.	f digg Joomla!
1995	JavaScript (after "Mocha") High-level, scripting, OO, imperative, functional. Designed (C, Java, Python, Scheme) for web programming (esp. client side)	Brendan Eich 	Dynamic web development, PDF docs, web browsers, widgets,	N Gmail Ps

▼ Arborele genealogic al limbajelor de programare



<http://amstel.science.uva.nl/~fotisg/python>

Fotis Georgatos <gef@ceid.upatras.gr>
Amstel Institute, Amsterdam, June 2002

Despre C++

▼ Istorie și origini

- Este unul din cele peste 2000 limbaje de programare.
- Originile C++
 - limbajele de programare de nivel scăzut (cod mașină, limbaje de asamblare)
 - limbajele de programare de nivel înalt (C, ADA, COBOL, FORTRAN)
 - programarea orientată pe obiecte
- 1979: Bjarne Stroustrup – Simula, C cu clase ... ->
- 1983: C++ (comparat cu C)
 - clase
 - moștenire
 - inlining (inline keywords și Class Member Functions)
 - argumente implicite pentru funcții
 - verificare puternică a tipurilor
 - funcții virtuale
 - supraîncărcarea funcțiilor
 - referințe cu simbolul &
 - cuvântul cheie `const`
- 1998: STL, C++98 - primele standarde C++
- 2003, 2005, 2011, 2014 - C++03, C++0x, C++11, C++14
- 2017 – o nouă revizuire

▼ Caracteristici

- ușor de programat (mai puțin timp de scris, mai scurt și mai ușor de citit); dezbateri în sală
- portabil - programele pot rula pe diferite calculatoare fără modificări sau cu puține modificări
- limbaj nesigur puternic tipizat (programatorul trebuie să știe exact ce trebuie să facă)
- acceptă atât variabile de tipuri definite explicit, cât și deduse
- acceptă verificarea statică (la compilare), dar și dinamică a tipurilor (în timpul execuției programului)
- oferă o gamă variată de paradigme de programare
- colecție foarte mare de biblioteci
- limbaj ce acceptă mai multe **paradigme de programare**:
 - generică: se pot folosi tipuri ce vor fi specificate mai târziu și vor fi instanțiate în funcție de necesități
 - structurată: *control intuitiv* (ordinea în care instrucțiunile sunt executate)
 - imperativă: *cum* (secvențe de instrucțiuni ce trebuie executate de calculator)
 - orientată pe obiecte: *obiecte + attribute + proceduri (metode)*
 - declarativă: *ce* (ce calcule trebuie făcute și nu cum să fie făcute)
 - funcțională: programele văzute ca evaluarea unor *funcții* matematice

1.2. Compilare și interpretare. Depanare. Preprocesare

Un fișier cod sursă poate fi compilat sau interpretat. Prin *compilare* codul sursă este translatat de un program (denumit compilator) în cod mașină, după care poate fi executat, iar prin *interpretare* un program este executat direct din cod sursă prin intermediul unui program (numit interpretor), care traduce și execută instrucțiunile cu instrucțiunile codului sursă.

▼ Diferențe între compilator și interpretor

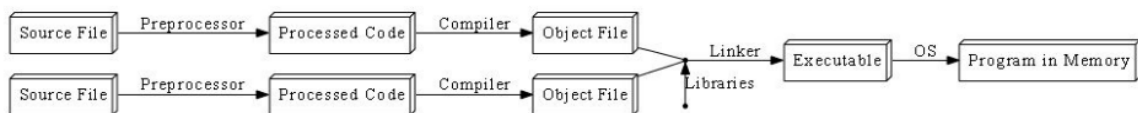
No	Compilator	Interpretor/Interpreter
1	Compilatorul ia întregul program ca intrare.	Interpretorul (Interpreterul) ia fiecare instrucțiune ca intrare. (El traduce instrucțiunile cu instrucțiunile.)
2	Se generează un cod obiect intermediar.	Nu se generează un cod obiect intermediar.
3	Instrucțiunile de control condiționale se execută mai rapid .	Instrucțiunile de control condiționale se execută mai încet .
4	Necesarul de memorie: Mai mare (deoarece este generat un cod obiect (object code))	Necesarul de memorie este mai redus.
5	Programul trebuie să fie compilat de fiecare dată, după ce se modifică sursa.	De fiecare dată un program de nivel înalt este convertit într-un program de nivel scăzut.
6	Erorile sunt afișate după verificarea întregului program .	Erorile sunt afișate pentru fiecare instrucțiune interpretată (dacă sunt)
7	Exemplu : C Compiler	Exemple : Interpretor BASIC

Explicație video aici: https://www.youtube.com/watch?v=_C5AHaS1mOA

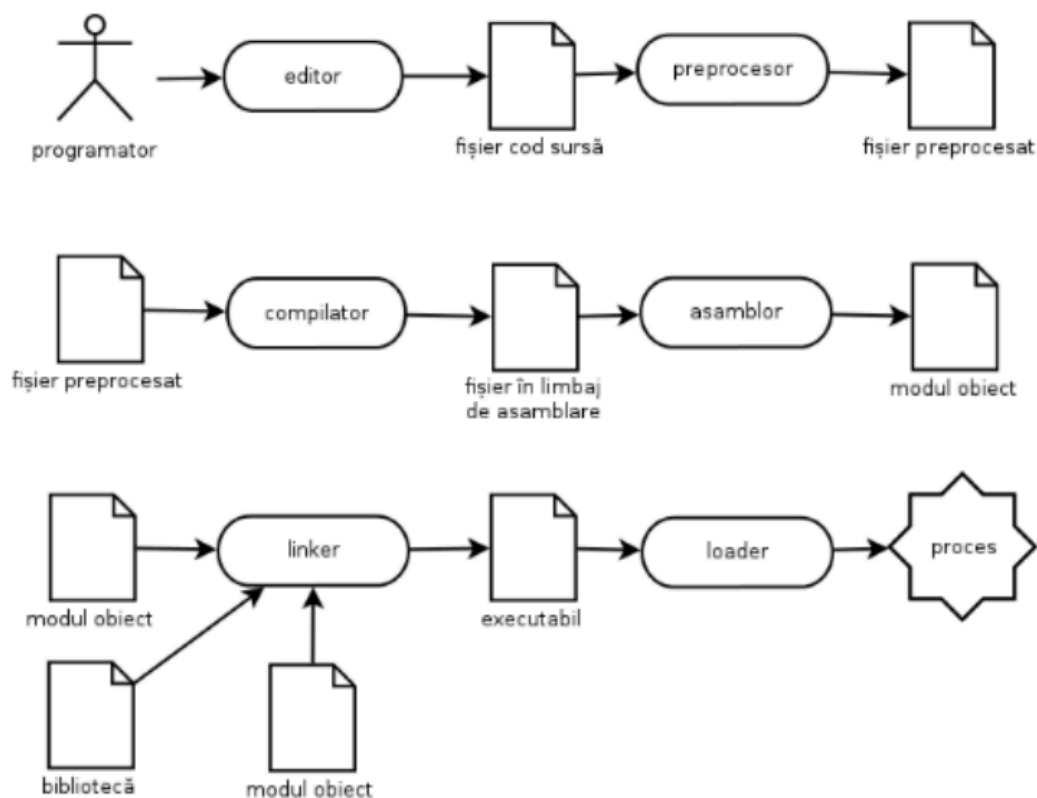
▼ Pașii compilării în C

1. **Preprocesarea**: Compilatorul elimină comentariile, înlocuiește macro-urile, copiază conținutul fișierelor incluse. Tot în această etapă se efectuează directivele de preprocesare `#define`. Astfel se obține codul procesat.
2. **Compilarea și asamblarea**: În această etapă codul este inițial transformat în cod de asamblare, iar apoi acesta în cod mașină specific mașinii pe care se compilează. După acest pas se obține fișierul obiect corespunzător codului (limbaj mașină) cu extensia `.o`, `.obj`. Codul generat în această etapă nu este încă executabil.
3. **Editarea de legături**: În acest pas toate fișierele obiect obținute în urma compilării sunt unite într-un singur fișier executabil. De asemenea sunt legate de fișierul executabil și toate bibliotecile externe incluse în cod (asociate fișierelor header). Tot acum are loc și transformarea adreselor simbolice în adrese reale. Un *linker* este un program care face legătura între fișierele obiect obținute în urma compilării și bibliotecile externe, obținând un fișier executabil. Bibliotecile pot fi legate în mai multe moduri: dinamic (când aplicația este executată, sistemul de operare face legătura cu librăriile necesare), static (fișierul executabil conține și codul librăriilor pe care le folosește) și întârziat (similar cu legarea în mod dinamic, dar bibliotecile sunt încărcate doar când aplicația are nevoie de ele (nu de la începutul execuției))

Schema compilării în C++



Schema detaliată a întregului proces de la scrierea codului sursă până la lansarea în execuție a programului (proces).Alfabet



▼ Depanare (debugging)

Putem avea mai multe erori în programul nostru, fie de compilare, fie de execuție. De aceea, este nevoie ca să depanăm programul, pentru a elimina aceste erori. La compilare putem avea mai multe erori, cum ar fi: încălcarea regulilor de **sintaxă**, **folosirea incorrectă** a tipurilor de date, alocării memoriei. De asemenea, putem avea erori în etapa de **editare de legături**. Prin *bug* înțelegem o eroare de programare, greu de sesizat. Putem avea, însă, și erori în timpul execuției programului, când acesta nu face ceea ce te aștepti să facă, ci altceva. Se spune că „un calculator/program face ceea ce i-ai spus să facă, nu ceea ce ai vrea să facă”. Prin *bug* înțelegem o eroare de programare, greu de sesizat, iar eliminarea bugurilor este o sarcină uneori dificilă.

▼ Forma unui program simplu în C++

```

/*comentariu; el nu influențează programul */
//directive preprocesare
#include <biblioteca> (Input/output, math, strings, ...)
#define
//declarații ale variabilelor
tipuri utilizator;
Variabile;
Funcții;
....
// definirea funcțiilor utilizator .....
//funcția principală
int main()
{
    .....
    return 0;
} //aici se încheie programul
  
```

▼ Exemplu

```

/* primul program in C++ */
// using namespace std;
#include <iostream>
int main ()
{
    std::cout << "Primul test 1, 2, 3. ";
    std::cout << "functioneaza.. \n";
    return 0;
}
  
```

Rezultatul va fi:

Primul test 1, 2, 3. functioneaza..

▼ Preprocesarea în C

Preprocesarea apare înaintea procesului de compilare a codului sursă (fișier text editat într-un editor și salvat cu extensia .c). Preprocesarea codului sursă asigură: includerea conținutului fișierelor (de obicei a fișierelor header), definirea de macro-uri (macrodefiniții) și compilarea condiționată.

1. **Directiva #include** copiază conținutul fișierului specificat în textul sursă. Are sintaxa: `#include <nume_fisier>` sau `#include "nume_fisier"`. În prima variantă se caută `nume_fisier` în directorul unde se află fișierele din biblioteca standard instalată odată cu compilatorul, iar în a doua variantă fișierul se caută în directorul curent (sau în subdirectoarele specificate ale acestuia). **Fișierele header** (cu extensia .h) sunt fișiere care conțin declarații de funcții și definiții de macro-uri (macrodefiniții). Ele sunt incluse în program folosind `#include`. Incluzând un fișier header într-un program, se va face legătura între program și fișierul header la editarea de legături, astfel putând folosi funcțiile din fișierul header fără a copia codul acestora. Utilitatea fișierelor header se observă mai ales când trebuie folosite aceleași funcții în mai multe fișiere de tip .c/.cpp. Există două tipuri de fișiere header: de sistem și scrise de utilizator. Cele scrise de utilizator se includ cu sintaxa `#include "fisier.h"`, iar cele de sistem cu sintaxa `#include <fisier.h>`.

▼ Example

```
#include <iostream>
#include <math.h>
#include <stdlib.h>
#include "biblioteca_mea.h"
```

2. **Directiva #define** este folosită pentru definirea (înlocuirea) constantelor simbolice și a macro-urilor. Definirea unei constante simbolice este un caz special al definirii unui macro (fragment de cod care primește un nume).

Sintaxa este `#define nume` sau `#define nume text`. Practic, în timpul preprocesării `nume` este înlocuit cu `text` (acest `text` poate să fie mai lung decât o linie, continuarea se poate face prin caracterul \ pus la sfârșitul liniei)

▼ Example

```
#define infinit 1000
#define pi 3.1415926
#define then
#define si &&
#define daca if
#define altfel else
#define max(x,y) x > y ? x : y
#define min(x,y) x < y ? x : y
#include <stdio.h>
int main()
{
    float a,b,c;
    printf("Dati lungimile laturilor:\n");
    scanf("%f %f %f",&a,&b,&c);
    daca ((a>=0) si (b>=0) si (c>=0) si (a<b+c) si (b<a+c) si (c<a+b))
        printf("%.2f, %.2f si %.2f formeaza triunghi.", a,b,c);
    altfel
        printf("Nu formeaza triunghi.");
    return 0;
}
```

▼ Macroui predefinite:

<u>DATE</u>	data compilării
<u>CDECL</u>	apelul funcției urmărește convențiile C
<u>STDC</u>	definit dacă trebuie respectate strict regulile ANSI C
<u>FILE</u>	numele complet al fișierului curent compilat
<u>FUNCTION</u>	numele funcției curente
<u>LINE</u>	numărul liniei curente


```
#include <stdio.h>
//constante simbolice
#define DEBUG
#define X -3
#define Y 5

int main()
{
    #ifdef DEBUG
        printf("Suntem in functia %s\n",__FUNCTION__); //main
    #endif
    #if X+Y
        double a=3.1;
    #else
        double a=5.7;
    #endif
    a*=2;
    #ifdef DEBUG
        printf("La linia %d valoarea lui a este %f\n",__LINE__,a); //18 6.2
    #endif
    a+=10;
    printf("a este %f",a); //16.2
    return 0;
}
```

3. **Directivele de compilare condiționată #if, #ifdef, #ifndef** facilitează dezvoltarea și în special testarea codului.

▼ Directiva #if

directiva `#if`:

```
#if expr
    text
#endif

#if expr
    text1
#else (elif)
    text2
#endif
```

- unde `expr` este o expresie constantă care poate fi evaluată de către preprocesor, `text`, `text1`, `text2` sunt porțiuni de cod sursă
- dacă `expr` nu este zero atunci `text` respectiv `text1` sunt compilate, altfel numai `text2` este compilat și procesarea continuă după `#endif`

▼ Directiva #ifdef

directiva `#ifdef`:

```
#ifdef nume      #ifdef nume
    text        text1
#else
    text2
#endif
```

- unde `nume` este o constantă care este testată de către preprocesor dacă este definită, `text`, `text1`, `text2` sunt porțiuni de cod sursă
- dacă `nume` este definită atunci `text` respectiv `text1` sunt compilate, altfel numai `text2` este compilat și procesarea continuă după `#endif`

▼ Directiva `#ifndef`

directiva `#ifndef`:

```
#ifndef nume      #ifndef nume
    text          text1
#else
    text2
#endif
```

- unde `nume` este o constantă care este testată de către preprocesor dacă NU este definită, `text`, `text1`, `text2` sunt porțiuni de cod sursă
- dacă `nume` NU este definită atunci `text` respectiv `text1` sunt compilate, altfel numai `text2` este compilat și procesarea continuă după `#endif`

▼ Incluziunea multiplă a modulelor

Directivele `#ifdef` și `#ifndef` sunt folosite de obicei pentru a evita incluziunea multiplă a modulelor în programarea modulară. Să considerăm că avem două fișiere header `a.h` și `b.h`. La începutul fiecărui fișier header se practică de obicei o astfel de secvență:

```
#ifndef _MODUL_A_
#define _MODUL_A_
...
#endif /* _MODUL_A_ */
```

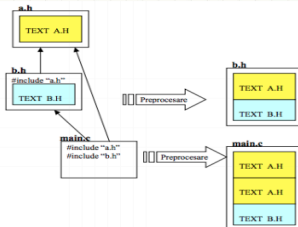
- directivele `#ifdef` și `#ifndef` sunt folosite de obicei pentru a evita incluziunea multiplă a modulelor în programarea modulară

- fișier antet "a.h"

- fișier antet "b.h"

- include pe "a.h"

- main include "a.h" și "b.h"



▼ Test de autoverificare 1 - Paradigme de programare, compilare, interpretare

Test 1 - Paradigme de programare, compilare și interpretare

Răspundeți corect la aceste întrebări simple, înainte de a continua parcurgerea materialului de curs.

dbxdrsl@gmail.com [Switch account](#)



* Indicates required question

Email *

Your email

Paradigma de programare în care se descrie logica unui calcul, fără să se prezinte modul de execuție, răspunzându-se la întrebarea CE? și nu la întrebarea CUM? se numește... * 3 points

Choose



Codul sursă este translatat de un program (denumit compilator) în cod mașină, după care poate fi executat. Este cazul * 3 points

Choose



În cazul unui compilator, care din următoarele afirmații sunt corecte? * 3 points

- ☐ Se ia fiecare instrucțiune ca intrare.
- ☐ Se generează un cod obiect intermediar.
- ☐ Necesarul de memorie: Mai mare (deoarece este generat un cod obiect (object code)).
- ☐ Instrucțiunile de control condiționale se execută mai încet.
- ☐ Erorile sunt afișate după verificarea întregului program.



1.3. Alfabetul și vocabularul limbajului de programare

1.3.1. Setul de caractere

Orice limbaj de programare are la bază un anumit alfabet. În majoritatea cazurilor setul de caractere este format din:

- literele alfabetului englez (de la A la Z, de obicei și majusculele, și literele mici, în total 52 de caractere);
- cifrele arabe (de la 0 la 9, în total 10 caractere);

- unele caractere speciale (, ; = < > # \$ % + - * / ' () etc.), a căror semnificație poate să difere de la un limbaj la altul.
- secvențe Escape / caractere speciale în C++: `\b` Backspace, `\t` Tab orizontal, `\v` Tab vertical, `\n` Linie nouă, `\f` Pagina nouă - formfeed `\r` Început de rând, `\"` Ghilimele, `\'` Apostrof, `\\` Backslash, `\?` Semnul întrebării, `\a` Alarmă

Există mai multe standarde de codificare a seturilor de caractere: [EBCDIC](#), [ASCII](#), [UTF-8](#).

▼ Detalii

- [EBCDIC](#) (Extended Binary Coded Decimal Interchange Code), un cod pe 8 biți, introdus de IBM;
- [ASCII](#) (American Standard Code for Information Interchange), introdus de ANSI (American National Standard Institute), este un cod pe 7 biți și permite codificarea a 128 de caractere (95 de caractere afișabile și 33 de caractere neafișabile, numite caractere de control). Ulterior, setul ASCII a fost extins la o codificare pe 8 biți, fiind disponibile astfel 256 de caractere. Primele 128 sunt din setul [ASCII](#) standard, iar următoarele 128 sunt coduri de caractere afișabile pentru caracterele unor alfabet naționale europene (francez, spaniol, român etc.), o parte din literele alfabetului grecesc, unele simboluri matematice, caractere speciale pentru desenat tabele etc. Ordonarea caracterelor alfabetului se face pe baza codurilor numerice corespunzătoare caracterelor respective. Tabela codurilor ASCII este...

▼ aici

ASCII Hex Symbol	ASCII Hex Symbol	ASCII Hex Symbol	ASCII Hex Symbol
0 0 NUL	16 10 DLE	32 20 (space)	48 30 0
1 1 SOH	17 11 DC1	33 21 !	49 31 1
2 2 STX	18 12 DC2	34 22 "	50 32 2
3 3 ETX	19 13 DC3	35 23 #	51 33 3
4 4 EOT	20 14 DC4	36 24 \$	52 34 4
5 5 ENQ	21 15 NAK	37 25 %	53 35 5
6 6 ACK	22 16 SYN	38 26 &	54 36 6
7 7 BEL	23 17 ETB	39 27 '	55 37 7
8 8 BS	24 18 CAN	40 28 (	56 38 8
9 9 TAB	25 19 EM	41 29)	57 39 9
10 A LF	26 1A SUB	42 2A *	58 3A :
11 B VT	27 1B ESC	43 2B +	59 3B ;
12 C FF	28 1C FS	44 2C ,	60 3C <
13 D CR	29 1D GS	45 2D -	61 3D =
14 E SO	30 1E RS	46 2E .	62 3E >
15 F SI	31 1F US	47 2F /	63 3F ?

ASCII Hex Symbol	ASCII Hex Symbol	ASCII Hex Symbol	ASCII Hex Symbol
64 40 @	80 50 P	96 60 `	112 70 p
65 41 A	81 51 Q	97 61 a	113 71 q
66 42 B	82 52 R	98 62 b	114 72 r
67 43 C	83 53 S	99 63 c	115 73 s
68 44 D	84 54 T	100 64 d	116 74 t
69 45 E	85 55 U	101 65 e	117 75 u
70 46 F	86 56 V	102 66 f	118 76 v
71 47 G	87 57 W	103 67 g	119 77 w
72 48 H	88 58 X	104 68 h	120 78 x
73 49 I	89 59 Y	105 69 i	121 79 y
74 4A J	90 5A Z	106 6A j	122 7A z
75 4B K	91 5B [	107 6B k	123 7B {
76 4C L	92 5C \	108 6C l	124 7C
77 4D M	93 5D]	109 6D m	125 7D }
78 4E N	94 5E ^	110 6E n	126 7E ~
79 4F O	95 5F _	111 6F o	127 7F

Unele coduri speciale sunt de interes pentru programatori:

- 7 (BELL), 10 LF (Line Feed), 13 CR (Carriage Return), 27 ESC (Escape), 32 Space,
- 33 '!', ..., 48 '0', ...,
- 65 'A', 66 'B', ...,
- 97 'a', 98 'b', ...
- [UTF-8 \(Unicode Transformation Format\)](#) este o codificare a caracterelor cu lățime variabilă, capabilă să codifice toate cele 1.112.064 caractere existente în Unicode, folosind unul până la patru unități de cod de 1 octet (8 biți). Codificarea este definită de Unicode Standard și a fost inițial proiectată de Ken Thompson și Rob Pike. Numele este derivat din formatul de transformare Unicode (sau Universal Coded Character Set) - 8 biți. Formatul a fost proiectat pentru o compatibilitate cu formatul existent [ASCII](#). Codurile cu valori numerice mai mici, care tind să apară mai frecvent, sunt codificate folosind mai puțini octeți. Primele 128 de caractere ale [Unicode](#), care corespund unu la unu cu cele din [ASCII](#), sunt codificate folosind un singur octet cu aceeași valoare binară ca [ASCII](#), astfel încât textul [ASCII](#) valabil este de asemenea un cod [UTF-8](#) valid. Deoarece octeții [ASCII](#) nu apar atunci când se codifică punctele de cod non-[ASCII](#) în [UTF-8](#), [UTF-8](#) este sigur de utilizat în majoritatea limbajelor de programare și documente care interpretează anumite caractere [ASCII](#) într-un mod special, cum ar fi `" / "` (*slash*) în nume de fișiere, `" \ "` (*backslash*) în secvențele de evadare și `" % "` în funcția `printf`.

1.3.2. Cuvinte cheie și cuvinte rezervate

Pe baza caracterelor ce alcătuiesc alfabetul limbajului se alcătuiesc cuvintele care formează vocabularul sau lexicul limbajului și cu ajutorul cărora se construiesc expresiile și instrucțiunile limbajului. Există două categorii de cuvinte și anume:

1. **cuvinte cheie** - acestea au un înțeles explicit într-un context precizat (de ex., în unele limbaje de programare cuvintele ce desemnează instrucțiuni pot fi folosite și ca nume de variabile, neexistând restricții; asemenea situații nu sunt însă indicate deoarece pot ascunde erori în logica programului și îl fac mai greu de înțeles);
2. **cuvinte rezervate** - acestea nu pot fi folosite decât în scopul pentru care au fost definite (de ex., în limbajul C++).

▼ Cuvinte rezervate în C

`auto`, `break`, `case`, `char`, `const`, `continue`, `default`, `do`, `double`, `else`, `enum`, `extern`, `float`, `for`, `goto`, `if`, `inline` (C99), `int`, `long`, `register`, `restrict` (C99), `return`, `short`, `signed`, `sizeof`, `static`, `struct`, `switch`, `typedef`, `union`, `unsigned`, `void`, `volatile`, `while`, `_Alignas` (C11), `_Alignof` (C11), `_Atomic` (C11), `_Bool` (C99), `_Complex` (C99), `_Generic` (C11), `_Imaginary` (C99), `_Noreturn` (C11), `_Static_assert` (C11), `_Thread_local` (C11) (acestea care încep cu simbolul `_` sunt utilizate prin intermediul macrodefinițiilor lor (din `stdalign.h`, `stdatomic.h`, `stdbool.h`, `complex.h`, `stdnoreturn.h`, `assert.h`, `threads.h`))

pentru preprocesor: `if`, `elif`, `else`, `endif`, `defined`, `ifdef`, `ifndef`, `define`, `undef`, `include`, `line`, `error`, `pragma`, `_Pragma`(C99), `asm`, `fortran`

▼ Cuvinte rezervate în C++

`alignas` (C++11), `alignof` (C++11), `and`, `and_eq`, `asm`, `atomic_cancel` (TM TS), `atomic_commit` (TM TS), `atomic_noexcept` (TM TS), `auto`(1), `bitand`, `bitor`, `bool`, `break`, `case`, `catch`, `char`, `char8_t` (C++20), `char16_t` (since C++11), `char32_t` (since C++11), `class`(1), `compl`, `concept` (C++20), `const`, `constexpr` (C++11), `constexpr` (C++20), `constinit` (C++20), `const_cast`, `continue`, `co_await` (C++20), `co_return` (C++20), `co_yield` (C++20), `decltype` (C++11), `default`(1), `delete`(1), `do`, `double`, `dynamic_cast`, `else`, `enum`, `explicit`, `export`(1)(3), `extern`(1), `false`, `float`, `for`, `friend`, `goto`, `if`, `inline`(1), `int`, `long`, `mutable`(1), `namespace`, `new`, `noexcept` (C++11), `not`, `not_eq`, `nullptr` (C++11), `operator`, `or`, `or_eq`, `private`, `protected`, `public`, `reflexpr` (reflection TS), `register`(2), `reinterpret_cast`, `requires` (C++20), `return`, `short`, `signed`, `sizeof`(1), `static`, `static_assert` (C++11), `static_cast`, `struct`(1), `switch`, `synchronized` (TM TS), `template`, `this`, `thread_local` (C++11), `throw`, `true`, `try`, `typedef`, `typeid`, `typename`, `union`, `unsigned`, `using`(1), `virtual`, `void`, `volatile`, `wchar_t`, `while`, `xor`, `xor_eq`

Semnificația numerelor din paranteze este: (1) - schimbare sau nou sens adăugat în C++11, (2) - schimbare în C++17, (3) - schimbare în C++20.

1.4. Sintaxa limbajului de programare

Și în cazul limbajelor de programare succesiunile de "cuvinte" construite după anumite reguli formează "propoziții", numite **instrucțiuni**. Sintaxa unui limbaj de programare reprezintă ansamblul de reguli prin care se determină dacă o anumită instrucțiune este alcătuită corect sau nu. Sintaxa unui limbaj de programare poate fi descrisă în diverse moduri, unul dintre acestea fiind notația **BNF** (**Backus-Naur Form**).

Notația BNF (**Backus-Naur Form**) a fost utilizată prima dată la descrierea sintaxei limbajului **ALGOL** (în cadrul raportului *ALGOL60* apărut în 1963) și este numită după doi dintre autorii acestui raport. În cadrul **BNF** sunt folosite metasimboluri, simboluri terminale și simboluri neterminale. Metasimboluri sunt simbolurile `<`, `>`, `|` și `::=` și ele fac parte din mecanismul de descriere a limbajului. Simbolul `|` semnifică o alternativă, simbolul `::=` înseamnă "se definește astfel". Simbolurile terminale sunt cuvinte care apar acolo unde se specifică în regulile de producție (de ex. `for`, `while`, `do`, `+`, `;` etc.). Simbolurile neterminale sunt încadrate în `<și>` și sunt definite prin producții ale limbajului (de ex., `<variabila>`, `<identificator>`, `<instrucțiune if>` etc.). În limbajul C++ sintaxa unui identificator se descrie în **BNF** astfel:

```
<identificator> ::= <litera> | <identificator> <cifra> | <identificator> <literă> |
```

unde:

```
<litera> ::= a | b | ... | z | A | B | ... | Z | _  
<cifra> ::= 0 | 1 | 2 | ... | 9
```

Simbolul `_` este considerat `<litera>` în acest context.

Conform acestei reguli, identificatorul are 3 definiții alternative: un identificator este fie o `<litera>`, fie un `<identificator>` urmat de o `<cifra>` sau o `<literă>` (definiție recursivă). Semnificația acestei definiții este următoarea: un identificator poate să conțină o singură literă, sau o literă urmată de oricâte litere și/sau cifre. Conform acestei definiții, sunt corecte sintactic următorii identificatori: `a`, `t1`, `sec12a1`. Descrierea sintaxei instrucțiunii condiționale `if-else` din limbajul C++ în notatie BNF este:

```
<instrucțiune if> ::=  
    if (<condiție>)  
        <instrucțiune1>  
    else  
        <instrucțiune2>
```

1.5. Semantica limbajului de programare

Semantica unei instrucțiuni reprezintă înțelesul pe care îl are acea instrucțiune, adică ce va executa calculatorul când o întâlnește. Astfel, pentru instrucțiunea `if-else` de mai sus, se va evalua condiția `<condiție>`, iar dacă aceasta este adevărată (în momentul evaluării), atunci se va executa o singură dată instrucțiunea `<instrucțiune1>`, respectiv, dacă este falsă, se va executa o singură dată instrucțiunea `<instrucțiune2>`.

La fiecare instrucțiune ce va fi descrisă în cadrul cursului se va prezenta semantica sa.

▼ Test de autoverificare 2 - Sintaxă, semantică

Test 2 - Sintaxă, semantică

Răspundeți corect la aceste întrebări simple, înainte de a continua parcurgerea materialului de curs.

dbxdrsl@gmail.com [Switch account](#)



* Indicates required question

Email *

Your email

Care din următoarele afirmații sunt adevărate? *

3 points

- ☐ Sintaxa reprezintă ansamblul de reguli prin care se determină dacă o anumită instrucțiune este alcătuită corect sau nu.
- ☐ Semantica se referă la înțelesul pe care îl au instrucțiunile, adică ce execută calculatorul când le întâlnește.

Care din următoarele afirmații sunt adevărate? *

3 points

- ☐ Notăția BNF se folosește pentru descrierea semanticii unui limbaj de programare.
- ☐ În C/C++, notația BNF pt. identificator poate fi: `<id>::=<cifră>|<id><cifra>|<id><literă>|<id>_`
- ☐ În limbajul C/C++ nu putem avea o variabilă denumită „else”.



1.6. Date

1.6.1. Variabile și constante

În cadrul programului, datele pot fi declarate ca fiind constante sau variabile. O constantă este o dată a cărei valoare nu se poate modifica pe parcursul execuției programului, deci rămâne constantă. O variabilă este o dată a cărei valoare se poate modifica pe parcursul execuției programului, deci ea poate varia, dar acest lucru nu este obligatoriu. Astfel, se poate declara o dată ca fiind variabilă în cadrul unui program, apoi ea să primească o anumită valoare, iar această valoare să rămână asociată respectivei variabile până la terminarea programului. Evident, atunci când se va declara o dată constantă, se va preciza și valoarea ei, iar când se va declara o dată variabilă, se subînțelege că ulterior, pentru a putea fi folosită, această variabilă va primi o anumită valoare. Majoritatea limbajelor de programare asignează o valoare inițială variabilelor, o dată cu declararea lor. Astfel, șirurile de caractere sunt inițializate la șirul vid, iar numerele sunt considerate cu valoarea zero. Firește, atât constantele cât și variabilele au o anumită structură, mai simplă sau mai complicată, și o anumită natură, dată de mulțimea valorilor posibile pentru o dată. Cu ele se pot face anumite operații, în funcție de natura și structura lor. Astfel, vom spune că o dată are un anumit tip. Există anumite convenții lexicale pentru scrierea constantelor și variabilelor:

- **ALL_CAPS** pentru constante
- **lowerToUpper** pentru variabile, cu nume cât mai sugestive

1.6.2. Expresii

Majoritatea limbajelor de programare definesc expresiile după un sistem de reguli sintactice, care, în general sunt următoarele:

1. orice constantă este expresie;
2. orice variabilă este expresie;
3. dacă E este expresie, atunci și (E) , $-E$, $+E$, $F(E)$ sunt expresii, unde F este numele unei funcții aplicabile expresiei E ;
4. dacă E_1 și E_2 sunt expresii, atunci și $E_1 + E_2$, $E_1 - E_2$, $E_1 * E_2$, E_1 / E_2 sunt expresii.

Acum, pe baza regulilor de mai sus putem construi expresii foarte complexe, pornind de la constante și variabile.

Astfel, să considerăm entitatea $(3+A) * (5 / (-B+C))$ și să verificăm dacă ea este expresie sau nu.

Să presupunem că A , B și C sunt variabile numerice întregi.

Cum 3 este constantă, conform regulii 1, ea este și expresie. A , fiind variabilă este, conform regulii 2 expresie.

Acum, conform regulii 4, $3+A$ este expresie, iar $(3+A)$ este tot expresie, conform regulii 3.

După simbolul înmulțirii (reprezentat adesea prin $*$), avem: 5 este expresie, fiind constantă, B , C ,

apoi $-B$ și $-B+C$ sunt expresii. În fine, conform regulii 3, $(-B+C)$ este tot expresie, apoi și

$(5 / (-B+C))$ este expresie, în conformitate cu regula 4, și, tot după această regulă, și $(3+A) * (5 / (-B+C))$ este expresie.

Datele sunt caracterizate de

- Tip
- Nume
- Valoare
- Adresă
- Domeniu de vizibilitate
- Timp de viață
- Operatori și apeluri de funcții

1.6.3. Variabile în C++

• Declararea variabilelor

```
tip variabila;  
tip var1, var2, ..., varn;  
tip variabila=expresie_constanta;
```

- **Variabile globale și variabile locale** Variabilele globale sunt declarate la începutul programului, în afara oricărei funcții (inclusiv în afara funcției `main`) și sunt vizibile oriunde în program (în cadrul oricărei funcții). Variabilele locale se declară în corpul funcției, înainte de a fi utilizate, iar ele sunt vizibile doar în acea funcție.

▼ Exemplificare

```
char c;  
signed char sc;  
int a, b;  
a = b = 5;  
int i;  
int suma = 0;  
long j;  
float x1, x2, x3;  
float pi = 3.14;  
double y;  
int patrat(int y)  
{  
    int x; x=y*y;  
    return x;  
}
```

Variabila x este caracterizată de tip = `int`, nume = x , valoare = 4 (de exemplu, dacă y este 2), domeniu de vizibilitate: funcția `patrat`, adresă (nu o putem ști, dar o putem determina cu `&x` și arată de ceva genul `0x7ffd3d518618`), timp de viață = pe durata unui apel al funcției.

- **Parametri pentru funcții** Aceste variabile sunt folosite în mod generic ca parametri pentru o funcție. Ele sunt vizibile doar în contextul funcției

▼ Explicație pe un exemplu

```
#include <iostream>  
  
int x=10,t;  
  
char egale(int x, int y)  
{  
    int a=1; t=x+y+a;  
    return x==y?'D':'N';  
}  
  
void main() {  
    int y=2,z=3; t=1;  
    cout<<t<<x<<y<<z;  
    if (egale(y,z)=='D' cout<<"Da"; else cout<<"NU";  
    cout<<t;  
}  
// 1,10,2,3,Nu (deoarece y->x, adica 2->x, z->y, adica 3->y și  
2!=3, apoi t=6
```

variabile globale, vizibile și în funcția `egale`

parametri formali ai funcției `egale`, vizibili doar în funcția `egale`

variabilă locală

variabile locale invizibile în funcția `egale`

Cum se accesează o variabilă globală când există o variabilă locală cu același nume?

Ce se întâmplă dacă vrem să accesăm variabila globală, când există o variabilă locală cu același nume?

Pentru a rezolva această problemă va trebui să folosim operatorul de rezoluție a domeniului.

Programul de mai jos explică cum se face acest lucru cu ajutorul operatorului de rezoluție a domeniului.

```
// Program C++ pentru a arata ca putem accesa o variabila globala
// folosind operatorul de rezolutie a domeniului :: cand
// exista o variabila locala cu acelasi nume

#include
using namespace std;

// x global
int x = 0;

int main()
{
    // Local x
    int x = 10;
    cout << "Valoarea x global este " << ::x;
    cout<< "\nValoarea x local este " << x;
    returneaza 0;
}
```

Iesire:
 Valoarea x global este 0
 Valoarea lui x local este 10

1.6.4. Atribuirea (asignarea) variabilelor

Prin instrucțiunea de atribuire (asignare) o variabilă primește valoarea unei expresii. Cu alte cuvinte, întâi se evaluează expresia, iar dacă evaluarea reușește, atunci variabila respectivă își pierde fosta valoare și primește valoarea evaluării expresiei. În contextul instrucțiunii de atribuire, dacă variabilele nu sunt de același tip, trebuie să fie considerate cazurile de compatibilitate și conversiile implicite care se fac (la tipul variabilei).

```
variabila = expresie;
SAU
Lvalues (left-side) = Rvalues (right-side)
unde Lvalues sunt variabile și Rvalues sunt expresii.
```

▼ Exemplu

În atribuirea `distance = rate * time;` avem: `Lvalue:` „distance” și `Rvalue:` „rate * time”.
 Contraexemple pt `Lvalue:` `5+5`, `"str"`, `const int i`.

1.7. Tipuri de date

Prin **tip de date** vom înțelege o mulțime de valori, împreună cu operațiile ce se pot executa cu ele. Fiecărei variabile, la declarare, i se va asocia un anumit tip. Tipul unei constante poate fi determinat implicit din valoarea constantei, sau poate fi precizat explicit ca în cazul variabilelor. Astfel, dacă constanta `K` are valoarea numerică 7, putem trage concluzia că ea este de tip întreg, sau de tip real, nu și logic sau șir de caractere. Totuși, există și limbaje în care se fac anumite convenții, de pildă că orice număr diferit de zero este considerat ca fiind cu valoarea de adevăr adevărat, iar numărul zero are valoarea de adevăr fals. Unele limbaje de programare (Basic, Python) permit declararea unor variabile fără a se preciza tipul lor, considerându-se astfel ca având un anumit tip general. Astfel, atunci când va fi folosită, variabila respectivă va fi considerată ca având cel mai adecvat tip cu putință, în situația concretă respectivă. De pildă, dacă este declarată o variabilă `X`, iar la un moment dat i se atribuie valoarea 3, atunci ea poate fi considerată ca având un tip numeric. Dacă ulterior, variabila `X` va primi valoarea "abc", adică un șir de caractere, se poate considera că `X` este de tip șir de caractere.

Pe baza constantelor și variabilelor se formează expresii. Bineînțeles, în formarea expresiilor se vor folosi acei operatori și acele funcții permise de tipurile valorilor asupra cărora se operează. Expresiile mici pot conduce la elaborarea de expresii mai mari, din ce în ce mai complexe.

Domeniul tipului (colecția de obiecte) este o **mulțime de valori** pentru care s-a adoptat un anumit **mod de reprezentare** în memorie. **Operațiile tipului** sunt operațiile ce se pot executa cu valorile de acel tip de date.

Avem următoarele categorii de tipuri de date:

- tipuri de date standard;
- tipuri de date structurate de nivel jos - operațiile se desfășoară la nivel de componentă;
- tipuri de date de nivel înalt - operațiile sunt implementate de algoritmi utilizator.

1.7.1. Tipuri de date standard în C++

- Tipuri caracter: tipurile `char`, `signed char`, `unsigned char`
- Tipuri întregi: tipurile caracter, întregi cu semn, întregi fără semn, tipurile enumerare
- Tipuri reale: tipurile întregi și tipurile flotante reale
- Tipuri aritmetice: tipurile întregi și cele flotante
- Tipuri de bază: caracter, întregi cu și fără semn, flotante
- Tipul void: desemnează o mulțime vidă

1.7.2. Reprezentarea în memorie a tipurilor de date (pe 32 biți)

Datele se reprezintă în sistem binar (baza 2), pe **biți**. 8 biți formează un **byte** sau **octet**.
 Există 10 tipuri de studenți: cei care înțeleg sistemul binar și cei care nu îl înțeleg.

▼ Reprezentarea datelor de tip `char`, `int`

char ch = 'a';
'a' are codul ASCII 97

Bitul de semn

0	1	1	0	0	0	0	1
---	---	---	---	---	---	---	---

$97 = 2^6 + 2^5 + 2^0$ (scrierea în baza 2 a lui 97)

$64 + 32$ $+ 1 = 97$

`int i = 190;`

0 1 0 1 1 1 1

Reprezentarea binară a lui 190 în memoria calculatorului

$190 = 2^7 + 2^5 + 2^4 + 2^3 + 2^2 + 2^1$ (scrierea în baza 2 a lui 190)

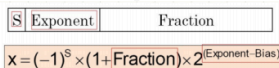
▼ Reprezentarea datelor de tip `float`

□ aproape jumătate din ele sunt în intervalul $[-1, 1]$

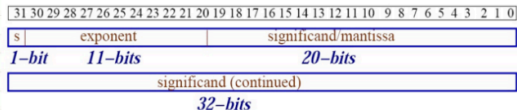


- 49,2% din numere reprezentate de float sunt în $[-1,1]$

▼ Reprezentarea datelor de tip `double`



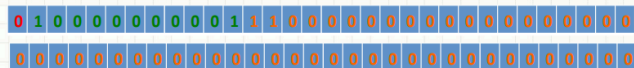
- **double**: ocupă 8 octeți = 64 biți, precizie dublă, bias = 1023



```
double d = 7.0; 7.0 = 1,75 * 4 = (-1)0 * (1+0.75) * 2(1025-1023)
```

Reprezentare binara exponent: $1025 = 1024 + 1 = 2^{10} + 2^0$

Reprezentare binara fractie: $0.75 = 0.5 + 0.25 = 2^{-1} + 2^{-2}$



1. **signed** - acesta este modificatorul implicit pentru toate tipurile de date; bitul cel mai semnificativ din reprezentarea valorii este semnul (0 - pozitiv, 1 - negativ);
2. **unsigned** - restricționează valorile numerice memorate la valori pozitive; domeniul de valori este mai mare deoarece bitul de semn este liber și participă în reprezentarea valorilor;
3. **short** - reduce dimensiunea tipului de date întreg la jumătate; se aplică doar pe întregi;
4. **long** - permite memorarea valorilor care depășesc limita specifică tipului de date; se aplică doar pe **int** sau **double** (la **int** dimensiunea tipului de bază se dublează, iar la **double** se mărește dimensiunea, de regulă cu doi octeți (de la 8 la 10 octeți))
5. **long long** - a fost introdus în **C99** pentru stocarea unor valori întregi de dimensiuni foarte mari

▼ Click aici pentru un tabel cu domeniile tipurilor de date (pe o anumită arhitectură cu un procesor pe 16 biți)

NUME TIP	DIMENSIUNE ÎN BITI	DOMENIU
unsigned char	8	0..255
char	8	-128..127
signed char	8	-128..127
unsigned int	16	0..65535
short int, signed int	16	-32768..32767
int	16	-32768..32767
unsigned long	32	0..4.294.967.295
long, (signed) long int	32	-2.147.483.648..2.147.483.647
float	32	Şase zecimale exacte
double	64	Zece zecimale exacte
long double	80	Zece zecimale exacte

La procesoarele pe 32 de biți, reprezentarea lui **int** este pe 32 de biți (4 octeți).

Reprezentarea numerelor pozitive diferă de cea a numerelor negative. Primul bit (numit și bit de semn) este folosit, în cazul numerelor pozitive pentru reprezentarea unor valori mai mari, pe când, în cazul numerelor negative, acest bit este folosit ca bit de semn. Adică, dacă el este 1, atunci se consideră că avem de a face cu un număr negativ, iar dacă e 0, cu un număr pozitiv. Pentru a înțelege reprezentarea, vom face o comparație între doi întregi, numerele 190 și -190, care adunate trebuie să dea 0. *Reprezentarea se numește în complement față de 2.*

▼ Reprezentare numere pozitive/negative

1. Reprezentarea numerelor 190 și -190:

- **signed int** – întreg cu semn (pozitiv sau negativ)
- **unsigned int** – întreg fără semn (pozitiv)

```
int i = 190;
```

0 1 0 1 1 1 1 1 0

Reprezentarea binară a lui 190 în memoria calculatorului

$190 = 2^7 + 2^5 + 2^4 + 2^3 + 2^2 + 2^1$ (scrierea în baza 2 a lui 190)

- cum se reprezintă -190 în memoria calculatorului?

1 0 1 0 0 0 0 1 0

Reprezentarea binară a lui -190 în memoria calculatorului

- care este logica unei asemenea reprezentări?

2. Suma în binar dintre 190 și -190:

- **signed int** – întreg cu semn (pozitiv sau negativ)
- **unsigned int** – întreg fără semn (pozitiv)

0 1 0 1 1 1 1 1 0

+

1 0 1 0 0 0 0 1 0

=

0 0

3. Suma dintre un număr pozitiv și corespondentul său negativ este zero:

- **signed int** – întreg cu semn (pozitiv sau negativ)
- **unsigned int** – întreg fără semn (pozitiv)

190
+
-190
=
0

Există anumite echivalențe între tipurile de date, după cum urmează: `signed short int` $\equiv$ `short`, `unsigned short int` $\equiv$ `unsigned short`, `signed int` $\equiv$ `int`, `unsigned int` $\equiv$ `unsigned`, `signed long int` $\equiv$ `long`, `unsigned long int` $\equiv$ `unsigned long`.

1.7.3. Tipuri de date derivate

Acestea se construiesc din obiecte, funcții și alte tipuri incomplete.

- tipul *tablou* de `T` (elementele de tip `T`)
- tipul *structură*
- tipul *uniune*
- tipul *funcție*, derivat din tipul returnat `T` și numărul și tipurile parametrilor (funcție ce returnează `T`)
- tipul *pointer*, derivat dintr-un tip referențiat (tip funcție, tip obiect, tip incomplet). Valorile tipului pointer sunt referințe la o entitate de tipul referențiat (pointer la `T`).
- Un tablou de dimensiune necunoscută, o structură sau uniune cu conținut necunoscut sunt tipuri incomplete

Tipurile aritmetice și tipul pointer se mai numesc și tipuri scalare. De asemenea, tablourile și structurile se numesc tipuri agregat.

1.8. Tipuri întregi

1.8.1. Constante întregi

- Zecimale
- Octale: au prefixul 0 (zero), de exemplu: `032` = 26 și `077` = 63
- Hexazecimale: au prefixul `0x` sau `0X`, de exemplu: `0x32` = 50 și `0x3F` = 63
- Întregi „long”: au sufixul `l` sau `L`, de exemplu: `2147483647L` și `0xaf9Fl` = 44959
- Întregi „unsigned” au sufixul `u` sau `U`, de exemplu: `345u` și `0xffffu` = 65535
- Caractere între apostrof: `'A'`, `'+'`, `'n'`
- Caractere în zecimal: `65`, `42`
- Caractere în octal: `'\101'`, `'\52'`
- Caractere în hexazecimal: `'\x41'`, `'\x2A'`
- Caractere speciale – secvențe escape

1.8.2. Operații și funcții pentru tipurile întregi

Operațiile pentru tipurile întregi sunt `+`, `*`, `%`, `==`, `!=`, `<`, `<=`, `>`, `>=`, `++`, `--`, iar **funcțiile** sunt cele de la tipul flotant și cele din bibliotecă: `tolower`, `toupper`, `isalpha`, `isalnum`, `isctrl`, `isdigit`, `isxdigit`, `islower`, `isupper`, `isgraph`, `isprint`, `ispunct`, `isspace`

▼ Tabel cu operatorii tipurilor întregi



Clasa de operatori	Operatori	Asociativitate
Unari	- (unar) ++ --	de la dreapta la stânga
Multiplicativi	* / %	de la stânga la dreapta
Aditivi	+ -	de la stânga la dreapta
Atribuire	=	de la dreapta la stânga
Relationali	< <= > >=	de la stânga la dreapta
De egalitate	== !=	de la stânga la dreapta
Atribuire și aritmetici binari	= *= /= %= += -=	de la dreapta la stânga

Operatorii speciali de incrementare și decrementare `++` și `--` se aplică doar unei expresii ce desemnează un obiect din memorie (L-value). De exemplu, `++5`; `--(k+1)`; `++i++` nu au sens, dar `(++i)++` are sens.

Expresie:	<code>++i</code>	<code>i++</code>	<code>--i</code>	<code>i--</code>
Valoare	<code>i+1</code>	<code>i</code>	<code>i-1</code>	<code>i</code>
i după evaluare	<code>i+1</code>	<code>i+1</code>	<code>i-1</code>	<code>i-1</code>

Operatorii de forma `a op= b` (`+=`, `-=`, `*=`, `/=`) au semnificația `a = a op b`.

1.9. Tipuri reale

Numerele reale sunt reprezentate în C/C++ prin tipurile `float` și `double`. De fapt, vorbim despre o reprezentare a numerelor raționale, deoarece numerele iraționale au un număr infinit de cifre, deci nu se pot reprezenta. `float` reprezintă numere reale în simplă precizie, reprezentate pe 4 octeți

(`sizeof(float) = 4`), $10^{-37} \leq \text{abs}(f) \leq 10^{38}$, 6 cifre semnificative), iar `double` reprezintă numere reale în dublă precizie, adică pe 8 octeți

(`sizeof(double) = 8`, $10^{-307} \leq \text{abs}(f) \leq 10^{308}$, 15 cifre semnificative). De asemenea, cu `long double` se pot reprezenta numere reale în „extra” dublă precizie, folosindu-se 12 octeți (`sizeof(long double) = 12`, $10^{-4931} \leq \text{abs}(f) \leq 10^{4932}$, 18 cifre semnificative). Limitele acestor domenii de reprezentare se găsesc în fișierul header `float.h`. Operațiile ce se pot face cu numere reale sunt: `+`, `-`, `*`, `/`, `==`, `!=`, `<`, `<=`, `>`, `>=`.

Constantele reale sunt implicit `double` și pot fi descrise astfel:

▼ vezi aici

```
125.435 1.12E2 123E-2 .45e+6 13. .56
1.12E2 = 1.12 x 102 = 112
123E-2 = 123 x 10-2 = 1.23
.45e+6 = 0.45 x 106 = 450000
13. = 13.00 și .56 = 0.56
```

Pentru a preciza că o constantă este de tip **float** și nu implicit **double**, constanta respectivă trebuie să aibă sufixul **f** sau **F**, de exemplu:

```
.56f 23e4f 45.54E-1F
23e4f = 23 x 104 = 230000.00
```

Pentru long double trebuie să aibă sufixul **l** sau **L**.

123.456e78L
 Funcții (în biblioteca **math.h**): **sin, cos, tan, asin, acos, atan, sinh, cosh, tanh, exp, log, log10, pow, sqrt, ceil, floor, fabs, ldexp, frexp, modf, fmod.**

1.10. Date booleene (logice)

Tipul bool reprezintă domeniul de valori: {**false, true**}, în care **false** = 0, iar **true** = 1, dar și orice întreg nenul. Operațiile care se pot face cu valorile logice sunt: **! && || == !=**

Declarații posibile:

```
bool x = 7; // x devine "true"
int y = true; // y devine 1
```

Expresii relaționale și expresii logice Acestea au forma următoare:

```
expresie_relativna ::= expr < expr | expr > expr | expr <= expr | expr >= expr | expr == expr | expr != expr
expresie_logica ::= ! expr | expr || expr | expr && expr
```

► Valoarea expresiilor relaționale

Negația logică: **! 0 = 1, ! orice_nr_diferit_de_0 = 0.**

Disjuncția logică (SAU) este reprezentată prin **||**, iar expresia rezultat este adevărată când cel puțin unul din expresiile operanzi este adevărată.

► Valoarea expresiilor logice **||**

Conjuncția logică (ȘI) este reprezentată prin **&&**, iar expresia rezultat este adevărată când ambele expresii operanzi sunt adevărate.

▼ Valoarea expresiilor logice **&&**

exp1	exp2	exp1 && exp2
= 0	nu se evaluează	0
<> 0	se evaluează	1, dacă exp2 <> 0 0, dacă exp2 = 0

▼ Exemple

O condiție de forma $a \leq x \leq b$ se scrie în limbajul C++ astfel:

```
(x >= a) && (x <= b)
(a <= x) && (x <= b)
```

O condiție de forma $a > x$ sau $x > b$ se scrie în limbajul C++ astfel:

```
(x < a) || (x > b)
!(x >= a && x <= b)
```

Legile lui De Morgan

- **! (A && B) este echivalent cu !A || ! B**
- **! (A || B) este echivalentă cu ! A && ! B**

1.11. Tipul void

Pentru situații speciale, se poate folosi tipul **void** (care semnifică mulțimea vidă), după cum urmează:

- Conversia în tip void a unei expresii semnifică faptul că valoarea sa este ignorată
- Utilizat pentru tipul pointer; nu se face controlul tipului la un pointer de tip void
- Utilizat pentru funcții fără valoare returnată sau pentru funcții fără parametri
- Este un tip incomplet ce nu poate fi completat
- Conversia în tip void a unei expresii semnifică faptul că valoarea sa este ignorată.

Observație. Tipul `void` este un tip incomplet ce nu poate fi completat.

1.12. Utilizare `typedef`

Typedef este un mecanism prin care se asociază un tip unui identificator:

▼ exemple

```
typedef char litera_mare;  
typedef short varsta;  
typedef int vector[20];  
typedef char string[30];  
typedef float matrice[10][10];  
typedef struct { double re, im; } complex;
```

Identificatorul respectiv se poate utiliza pentru a declara variabile:

▼ exemple

```
litera_mare u, v='a';  
varsta v1, v2;  
vector x; string s;  
matrice a;  
complex z;
```

sau funcții

▼ exemple

```
complex suma(complex z1, complex z2) {  
    complex z;  
    z.re=z1.re+z2.re; z.im=z1.im+z2.im;  
    return z;  
}
```

1.13. Operatori speciali

1.13.1. Operatorul condițional `?:`

▼ Detalii

Sintaxa: `exp1 ? exp2 : exp3` **Semantica**

- Se evaluează `exp1`
- Dacă `exp1` are valoare `true` (nenulă), atunci valoarea expresiei este valoarea lui `exp2`, iar `exp3` nu se evaluează
- Dacă `exp1` are valoare false (nulă), atunci valoarea expresiei este valoarea lui `exp3`; `exp2` nu se evaluează

Operatorul `?:` este drept asociativ

Exemple

```
x >= 0 ? x : y  
x > y ? x : y  
x > y ? x > z ? x : z : y > z ? y : z  
joc=(raspuns=='1')?JocSimplu();JocDublu();  
#include <iostream>  
using namespace std;  
void main(){  
    int a=1, b=2, c=3;  
    int x, y, z;  
    x = a?b:c?a:b;  
    y = (a?b:c)?a:b; /* asociere stanga */  
    z = a?b:(c?a:b); /* asociere dreapta */  
    cout<< "x=" << x << "\ny=" << y << "\nz=" << z;  
}  
/* x=2 y=1 z=2 */
```

1.13.2. Operatorul virgulă `,`

▼ Detalii

Sintaxa: `expresia_virgula ::= expresie, expresie` **Semantica**

- Se evaluează prima expresie apoi cea de-a doua.
- Valoarea și tipul întregii expresii este valoarea și tipul operandului drept.
- Operatorul virgulă are cea mai mică precedență.

Exemplu

```
a = 1, b = 2;  
i = 1, j = 2, ++k + 1;  
k != 1, ++x * 2.0 + 1;  
for(suma = 0, i = 1; i <= n; suma += i, ++i);
```

1.13.3. Operatorul `sizeof()`

`sizeof()` este un operator unar, sub forma unei funcții, ce permite determinarea numărului de octeți pe care se reprezintă un obiect (un tip sau o expresie).

▼ Exemple:

```
sizeof(int), sizeof(double);
sizeof(b*b-4*a*c), sizeof(i);
sizeof(char)<=sizeof(short)<=sizeof(int)<=sizeof(long)
sizeof(signed)=sizeof(unsigned) = sizeof(int)
sizeof(float)<=sizeof(double)<=sizeof(long double)
```

▼ Exemplu de utilizare

```
#include <iostream>
using namespace std;
void main(){
    int x = 1; double y = 9; long z = 0;
    cout << "Operatorul sizeof()\n\n\n";
    cout << "sizeof(char) = " << sizeof(char) << endl;
    cout << "sizeof(int) = " << sizeof(int) << endl;
    cout << "sizeof(short) = " << sizeof(short) << endl;
    cout << "sizeof(long) = " << sizeof(long) << endl;
    cout << "sizeof(float) = " << sizeof(float) << endl;
    cout << "sizeof(double) = " << sizeof(double) << endl;
    cout << "sizeof(long double) = " << sizeof(long double) << endl;
    cout << "sizeof(x + y + z) = " << sizeof(x+y+z) << endl;
}
```

Rezultatul executiei ar putea fi:

```
Operatorul sizeof()
sizeof(char) = 1
sizeof(int) = 4
sizeof(short) = 2
sizeof(long) = 4
sizeof(float) = 4
sizeof(double) = 8
sizeof(long double) = 8
sizeof(x + y + z) = 8
```

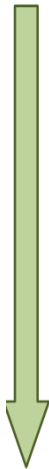
1.13.4. Alți operatori

- operatorul de acces la elementele tabloului `[]`: `int a[100]; a[3]=17;`
- operatorul de apel de funcție `()`: `y = f(x);`
- operatorul adresă `&` și operatorul de „dereferențiere” `*` (strâns legat de pointeri (vom reveni)):
 - `int a, *p; // p este un pointer la int`
 - `p = &a; // p este pointer la a`
 - `*p = 3; // valoarea lui a devine 3`

1.14. Prioritatea și asociativitatea operatorilor.

Tabel cu prioritatea și asociativitatea operatorilor

▼ click aici



Nr.	Clasa de operatori	Operatori	Asociativitate
1.	Primari	<code>() [] . -> ::</code>	de la stânga la dreapta
2.	Unari	<code>! ~ ++ -- sizeof(tip) -(unar)</code> <code>*(dereferențiere) &(referențiere)</code>	de la stânga la dreapta
3.	Multiplicativi	<code>* / %</code>	de la stânga la dreapta
4.	Aditivi	<code>+ -</code>	de la stânga la dreapta
5.	Deplasare pe bit	<code><< >></code>	de la stânga la dreapta
6.	Relaționali	<code>< <= > >=</code>	de la stânga la dreapta
7.	De egalitate	<code>== !=</code>	de la stânga la dreapta
8.		<code>& (SI logic pe bit)</code>	de la stânga la dreapta
9.		<code>^ (XOR pe bit)</code>	de la stânga la dreapta
10.		<code> (SAU logic pe bit)</code>	de la stânga la dreapta
11.		<code>&&</code>	de la stânga la dreapta
12.		<code> </code>	de la stânga la dreapta
13.	Condițional	<code>?:</code>	de la dreapta la stânga
14.	De atribuire	<code>= += -= *= %= &= ^= = <<= >>=</code>	de la dreapta la stânga
15.	Virgula	<code>,</code>	de la stânga la dreapta

1.15. Conversii între tipuri

1.12.1. Reguli pentru conversia implicită

În absența unui `unsigned`, obiectele se convertesc la tipul cel mai "înalț" în ordinea (descrescătoare): `long double`, `double`, `float`, `long int`, `int`. Regulile de conversie pentru operanzii `unsigned` depind de implementare. Conversia la `unsigned` se face doar în caz de necesitate (de ex. valoarea din `unsigned` nu "încapă" în celălalt operand). Regula "integer promotion" spune că operațiile se fac cel puțin în tipul `int`, deci `char` și `short` sunt "promovați" la `int`.

La o asignare (`v = exp`) tipul membrului drept se convertește la tipul membrului stâng (care este și tipul rezultatului).

Atenție! Prin conversiile implicite între tipuri, se pot produce:

- pierderea preciziei (`double ->float ->long int`)
- pierderea unor biți semnificativi (`long ->int`)
- nedeterminări

▼ Exemplu conversii implicite

```
#include <iostream>
using namespace std;
int main(void){
    char c1 = -126, c2;          /* c1 = 10000010 */
    unsigned char c3, c4 = 255; /* c4 = 11111111 */
    short s1, s2 = -32767; /* s2=10000000 00000001 */
    short s3 = -1, s4;          /* s3 = 11111111 11111111 */
    s1 = c1;
    cout << "c1=" << (int)c1 << " s1=" << s1 << endl;
    c2 = s2;
    cout << "c2=" << (int)c2 << " s2=" << s2 << endl;
    c3 = s3;
    cout << "c3=" << (int)c3 << " s3=" << s3 << endl;
    s4 = c4;
    cout << "c4=" << (int)c4 << " s4=" << s4 << endl;
    return 0;
}
```

▼ Rezultatul execuției

```
c1 = 10000010      s1 = c1;
c1 = -126, s1 = -126
```

```
s2=10000000 00000001  c2 = s2;
c2 = 1, s2 = -32767
```

```
s3 = 11111111 11111111  c3 = s3;
c3 = 255, s3 = -1
```

```
c4 = 11111111      s4 = c4; char c1 = -126, c2;
c4 = 255, s4 = 255 unsigned char c3, c4 =
                    255;
                    short s1, s2 = -32767;
                    short s3 = -1, s4;
```

1.12.2. Forțarea tipului - *cast*

Conversia explicită la tipul `numetip` se face prin: `(numetip) expresie`.

▼ Exemple:

```
(long)('A' + 1.0)
(int)(b*b-4*a*c)
(double)(x+y)/z
(float)x*y/z
x / (float)2
```

Exemplu de utilizare *cast*

```
#include <iostream>
using namespace std;
int main(void){
    int i, j; double x, y, z, t;
    i = 5/2; x = 5/2;
    y = (double)(5/2); j = (double)5/2;
    z = (double)5/2; t = 5./2;
    cout << i << ", " << x << ", ";
    cout << y << ", " << j << ", ";
    cout << z << ", " << t << ", " << endl;
    return 0;
}
/* 2, 2, 2, 2, 2.5, 2.5 */
```

Alt exemplu:

```
int a=1, b=2;
media = (a+(float)b)/2; // media devine 1.5
media = (a+b)/2; // media devine 1 !!!!
```


1.13. Fișiere în biblioteca standard, relative la tipuri

`<limits.h>`

- pentru tipurile întregi
- întregul min/max: `INT_MIN`, `INT_MAX`
- Numărul de biți pe caracter `CHAR_BIT`

`<float.h>`

- pentru tipurile flotante
- exponentul maxim
- precizia zecimală etc.

`<stdlib.h>`

- conține funcții de conversie
- șir de caractere în int: `atoi(const char*)`
- șir de caractere în float: `atof(const char*)`

▼ Test 3 - Date, tipuri de date

Test 3 - Date, tipuri de date

Răspundeți corect la aceste întrebări simple, înainte de a continua parcurgerea materialului de curs.

dbxdrgsl@gmail.com [Switch account](#)



Not shared

* Indicates required question

Care din următoarele afirmații sunt adevărate? *

3 points

- ☐ Variabilele primesc o valoare la începutul programului, când sunt declarate, apoi nu-și mai pot modifica acea valoare deloc.
- ☐ În limbajul C, expresia $(3+A)(5/(-B+C))$ nu este corectă.
- ☐ Variabilele sunt caracterizate doar de nume și valoarea pe care o au la un moment dat.

Presupunând că x, y și z sunt trei variabile de tip „int”, care din următoarele atribuiri sunt corecte?

* 3 points

- ☐ $x=x+1$
- ☐ $x+1=x;$
- ☐ $x+=2;$
- ☐ $x==y+2-z;$
- ☐ $x=x+y-z;$
- ☐ $x=x+2;$

Care din următoarele afirmații nu sunt corecte? *

3 points

- ☐ Datele de tip „unsigned char” se reprezintă pe 7 biți.
- ☐ Datele de tip „char” se reprezintă pe 8 biți.
- ☐ „short” reduce dimensiunea tipului de date întreg la jumătate și se aplică doar pe întregi.
- ☐ Constantele octale au prefixul O (litera „O”), de exemplu: $O32 = 26$ și $O77 = 63$.
- ☐ Expresia $++++$ nu are sens, dar $(++)++$ are sens.
- ☐ $!(A \&\& B)$ este echivalent cu $!A \&\& !B$
- ☐ $!(A \parallel B)$ este echivalentă cu $!A \&\& !B$
- ☐ `sizeof` este un mecanism prin care se asociază un tip unui identificator.
- ☐ Operatorii de deplasare pe biți sunt asociativi de la stânga la dreapta.
- ☐ Operatorul `=` este asociativ de la stânga la dreapta.



1.16. Operații pe biți

În interiorul procesorului, toate operațiile matematice sunt făcute prin modificări la nivel de biți. În limbajul C, avem acces la operatorii binari, astfel putând efectua operații pe biți.

1.14.1. Operatori logici pe biți

Operatorii logici pe biți sunt: `&` (AND), `|` (OR), `^` (XOR), `~` (NOT)

▼ detalii

AND: operatorul `&`

Bit 1	Bit 2	Bit rezultat
0	0	0
0	1	0
1	0	0
1	1	1

Operatorul AND este un operator binar aplicat pe două șiruri de lungime egală de biți. Dacă biții de pe aceeași poziție sunt ambii 1, bitul rezultat este 1, altfel este 0.

▼ Exemplu

6 = 110(binar) 2 = 010(binar)

110 &

010

010

6&2 = 2

OR: operatorul `|`

Bit 1	Bit 2	Bit rezultat
0	0	0
0	1	1
1	0	1
1	1	1

Operatorul OR este un operator binar aplicat pe două șiruri de lungime egală de biți. Dacă biții de pe aceeași poziție sunt ambii 0, bitul rezultat este 0, altfel este 1.

▼ Exemplu

6 = 110(binar) 2 = 010(binar)

110 |

010

110

6|2 = 6

XOR: operatorul `^`

Bit 1	Bit 2	Bit rezultat
0	0	0
0	1	1
1	0	1
1	1	0

Ca și OR și AND, XOR este un operator binar aplicat pe două șiruri de lungime egală de biți. Dacă biții de pe aceeași poziție au aceeași valoare, bitul rezultat este 0, altfel este 1.

▼ Exemplu

6 = 110(binar) 2 = 010(binar)

110 ^

010

100

6^2 = 4

NOT : operatorul `~`

Bit 1	Bit rezultat
0	1
1	0

NOT se aplică unui șir de biți și inversează valoarea fiecărui bit în parte.

▼ Exemplu

$$6 = 110(\text{binar})$$

$$\sim 110$$

$$001$$

$$\sim 6 = 1$$

Asadar, tabelele pentru operatorii pe biți sunt următoarele:

X	Y	X&Y	X Y	X^Y	~(X)
0	0	0	0	0	1
0	1	0	1	1	1
1	0	0	1	1	0
1	1	1	1	0	0

1.14.2. Operatorii de deplasare pe biți

Operatorii de deplasare pe biți sunt `<<` și `>>`. În exemplele următoare vom considera că un număr este reprezentat pe 1 octet (byte) (8 biți). În realitate, reprezentarea unui număr depinde atât de sistemul folosit, cât și de tipul de variabilă (`int`, `long`, `unsigned int`, etc).

▼ detalii

Deplasarea ("shiftare") la stânga `<<`

Operatorul de "shiftare" la stânga mută toți biții la stânga cu numărul specificat de poziții și umple restul de poziții rămase goale cu 0. Operatorul de "shiftare" la stânga este echivalent cu înmulțirea numărului cu 2 la puterea numărului de poziții deplasate ("shiftate").

▼ Exemplu

```
a = 5 (00000101 în reprezentarea binară pe un byte)
a << 1 = 10 (00001010 în binar) (echivalent cu 5*2)
a << 2 = 20 (00010100 în binar) (echivalent cu 5*4)
a << 3 = 40 (00101000 în binar) (echivalent cu 5*8)
```

Atenție însă la limita reprezentării numărului! Astfel o "shiftare" cu 6 biți va duce la un rezultat diferit de ce ne așteptăm (320). `a << 6 = 64` (01000000 în binar) - primul bit de 1 s-a pierdut

Deplasarea ("shiftare") la dreapta `>>`

Operatorul de "shiftare" la dreapta mută toți biții la dreapta cu numărul specificat de poziții și umple restul de poziții rămase goale cu 0. Operatorul de shiftare la dreapta este echivalent cu împărțirea numărului la 2 la puterea numărului de poziții "shiftate".

▼ Exemplu

```
a = 5 (00000101 în reprezentarea binară pe un byte)
a >> 1 = 2 (00000010 în binar) (echivalent cu 5/2 (împărțire întreagă))
a >> 2 = 1 (00000001 în binar) (echivalent cu 5/4)
a >> 3 = 0 (00000000 în binar) (echivalent cu 5/8)
```

Atenție: Deoarece în cazul numerelor cu semn, primul bit din reprezentare este bitul de semn, folosirea operatorilor de "shiftare" în cazul numerelor negative duce la rezultate greșite.

```
a = -5 (10000101 în reprezentarea binară cu semn pe un byte)
a >> 1 = 66 (01000110 în reprezentarea binară cu semn pe un byte)
```

▼ Test 4 - Operatori pe biți

Test 4 - Operatori pe biți

Răspundeți corect la aceste întrebări simple, înainte de a continua parcurgerea materialului de curs.

dbxdrsl@gmail.com [Switch account](#)



Not shared

* Indicates required question

Pentru care operator pe biți este adevărată tabela de valori din imaginea alăturată? * 3 points

Bit 1	Bit 2	Bit rezultat
0	0	0
0	1	0
1	0	0
1	1	1

Choose ▼

Fie variabila „a” cu valoarea 5. Care este valoarea lui „(a<<3)>>2|4”? * 3 points

Choose ▼

Submit

[Clear form](#)

Google Forms This content is neither created nor endorsed by Google.



Bibliografie

- www.cplusplus.com
- www.tiobe.com/
- en.cppreference.com/
- www.greenteapress.com/thinkcpp/index.html
- [ocw.mit.edu/courses/ electrical-engineering-and-computer-science/6-096-introduction-to-c-january-iap-2011/](http://ocw.mit.edu/courses/electrical-engineering-and-computer-science/6-096-introduction-to-c-january-iap-2011/)
- [kickassinfographics.com/history/ the-evolution-of-computer-languages-infographic/](http://kickassinfographics.com/history/the-evolution-of-computer-languages-infographic/)